

Phase 1 - Week 1 - Lesson Day 4

NumPy Applied + Week 1 Mini Challenge

Today's Objectives

This is the final lesson day of Week 1. Today adds three more practical NumPy tools - random number generation, combining arrays, and basic linear algebra - then wraps everything from the week into one combined mini challenge.

- Generate random numbers and understand why a random seed matters
- Combine and stack arrays together
- Understand the dot product and basic matrix multiplication
- Apply everything from Week 1 in one combined mini challenge

1. Random Number Generation

Definition: NumPy's **random** module generates random numbers. This might sound like a small feature, but it is essential in data science for simulating data, generating test datasets, and later for splitting real datasets into training/testing portions and initializing machine learning models with random starting values.

```
np.random.seed(42)    # locks in a fixed random sequence so results are reproducible

random_ints = np.random.randint(1, 100, size=10)    # 10 random integers between 1 and 99
random_floats = np.random.rand(5)                  # 5 random floats between 0 and 1
random_normal = np.random.normal(loc=50, scale=10, size=100) # 100 values, normal distribution
```

Why `np.random.seed()` matters: without setting a seed, you get a different set of random numbers every single time you run your code. Setting a seed makes the "random" output reproducible - the same seed always produces the same sequence. This matters constantly in data science: if you and Halimo both run the same code with the same seed, you should get identical results, which makes debugging and comparing work together much easier.

`randint(low, high, size)` gives whole numbers. **`rand(size)`** gives decimals between 0 and 1. **`normal(loc, scale, size)`** gives values from a bell-curve distribution, where **`loc`** is the center (mean) and **`scale`** is how spread out the values are (standard deviation).

2. Combining and Stacking Arrays

Definition: These functions let you join multiple arrays together into one, in different directions. This is useful whenever you have data coming from separate sources that needs to be combined into a single dataset.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

```
print(np.concatenate([a, b]))    # [1, 2, 3, 4, 5, 6] - joins 1D arrays end to end

print(np.vstack([a, b]))
# [[1, 2, 3],
#  [4, 5, 6]]    -> stacks vertically, creating new ROWS

print(np.hstack([a, b]))         # [1, 2, 3, 4, 5, 6] - stacks horizontally
```

A simple way to remember the difference: **vstack** stacks arrays on top of each other like adding new rows to a table. **hstack** lines them up side by side. For 1D arrays, hstack and concatenate behave the same way - the difference becomes more important once you're combining 2D arrays (tables) together.

3. Linear Algebra Basics - Dot Product & Matrix Multiplication

Definition: The *dot product* multiplies the corresponding elements of two arrays together, then adds up all the results into a single number. *Matrix multiplication* extends this same idea to full matrices, combining the rows of one matrix with the columns of another. This might feel abstract now, but it is the exact mathematical operation that powers how linear regression and neural networks make predictions later in the course.

Dot product of two 1D arrays:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.dot(a, b))
# (1*4) + (2*5) + (3*6) = 4 + 10 + 18 = 32
```

Matrix multiplication of two 2D arrays:

```
matrix_a = np.array([[1, 2],
                     [3, 4]])
matrix_b = np.array([[5, 6],
                     [7, 8]])

print(np.dot(matrix_a, matrix_b))
# each value in the result comes from multiplying a ROW of matrix_a
# by a COLUMN of matrix_b, and summing - exactly like the dot product above,
# repeated for every row/column pair
```

For matrix multiplication to work, the number of **columns** in the first matrix must match the number of **rows** in the second matrix - otherwise NumPy will raise a shape error.

Today's Exercise

Open the accompanying notebook file **day-4-numpy-applied.ipynb** and complete the exercises below. Part A and B cover today's new material; Part C is the Week 1 Mini Challenge, which combines everything learned across all four lesson days this week.

Part A - Random Numbers & Stacking (4 questions)

1. Reproducible random integers

Set a random seed of your choice, then generate 15 random integers between 1 and 100. Run the cell twice and confirm you get the same numbers both times.

2. Simulate normal data

Generate 50 random values from a normal distribution with a mean of 70 and a standard deviation of 8 (think: exam scores). Print the mean and standard deviation of your generated data using aggregate functions from Day 3.

3. Stack two arrays vertically

Create two 1D arrays of 4 numbers each, and stack them vertically into a 2x4 matrix.

4. Stack two arrays horizontally

Using the same two arrays from Question 3, stack them horizontally instead and compare the shape to Question 3's result.

Part B - Dot Product & Matrix Multiplication (2 questions)

5. Dot product by hand vs NumPy

Given $a = [2, 4, 6]$ and $b = [1, 3, 5]$, calculate the dot product manually with pen and paper first, then verify your answer using `np.dot()`.

6. Matrix multiplication

Create two 2x2 matrices of your choice, multiply them using `np.dot()`, and print the result.

Part C - Week 1 Mini Challenge (Simulated Test Scores)

This challenge combines Day 2 (arrays), Day 3 (operations, aggregates, masking), and Day 4 (random generation) into one connected task. No loops allowed anywhere - vectorized operations only.

1. Set a random seed of your choice for reproducibility.
2. Simulate 100 random "test scores" between 0 and 100 using `np.random.randint()`.
3. Calculate and print the mean, standard deviation, minimum, and maximum of the scores.
4. Using boolean masking, create a new array containing only the students who "passed" (score > 60).
5. Print how many students passed, using `.sum()` on a boolean mask (no loop).
6. Print what percentage of students passed, calculated from your results in Steps 2 and 5.

How to Approach It

For the mini challenge, work through it in the exact order listed - each step builds on the previous one's output. If Step 4 or 5 feels tricky, re-read the Boolean Masking section from Day 3 before attempting it; the pattern is identical, just applied to your own simulated data instead of a given array.

Once you and Halimo have completed all three parts, bring your answers back to the lesson chat for review. This wraps up Week 1 - next up is Week 2: Pandas Mastery.